

Sort it Like You Mean It: Discovering Semantically Interesting Attribute Augmentations to Sort Tables

Akash Khatri
University of Utah
akash.m.khatri@utah.edu

Mir Mahathir Mohammad
University of Utah
mahathir.mohammad@utah.edu

El Kindi Rezig
University of Utah
elkindi.rezig@utah.edu

ABSTRACT

Sorting is a fundamental operation in table analysis. Data scientists frequently sort tables to uncover key insights—for example, identifying the top 10 products by sales. However, this process is largely manual. Data scientists must (1) understand the semantics behind the sorting they wish to apply, and (2) ensure the necessary attributes are present—often requiring manual augmentation of the table. But what if data scientists could receive suggestions for semantically meaningful ways to sort a table, powered by automatic augmentations from a data lake? In this demo, we present INSIGHTSORT, an end-to-end system that recommends attribute augmentations to enable richer, more insightful sorting for table exploration. INSIGHTSORT works by: (1) discovering potential augmentations by linking the input table with relevant data lake tables, and (2) leveraging a Large Language Model (LLM) to synthesize the top-k diverse sorting attributes based on their semantics. A companion video is available at [1].

PVLDB Reference Format:

Akash Khatri, Mir Mahathir Mohammad, and El Kindi Rezig. Sort it Like You Mean It: Discovering Semantically Interesting Attribute Augmentations to Sort Tables. PVLDB, 18(12): 5427-5430, 2025.
doi:10.14778/3750601.3750688

1 INTRODUCTION

Sorting is a critical operation in both data management and exploratory analysis. For instance, when analyzing a table of employee records, sorting by attributes such as income or seniority can surface meaningful patterns or generate insightful reports. This might reveal, for example, employees who have stayed with the company for many years but have received relatively few raises—highlighting potential disparities or areas for further investigation.

However, this process assumes that users already know which attributes to sort by and that these attributes are readily available in the table. It also requires users to understand the semantics of the desired sorting logic—for example, recognizing that sorting employees by years of seniority may involve computing tenure from hire dates.

Data lakes—whether organizational or public [8]—are becoming increasingly ubiquitous, offering opportunities for data augmentation for several tasks [5]. We propose an approach to identify

candidate sorting attribute sets within a data lake to augment a given input table. This enables the table to be sorted in semantically meaningful ways that go beyond its original schema.

Even when users have a general sense of the sorting semantics they want to apply, distinguishing between subtle variations in sorting strategies can be challenging. For example, if users wish to sort countries by “economic performance,” potential sorting attributes might include GDP, GDP per capita, GDP growth rate, or inflation rate. Each of these captures a different facet of economic performance, and identifying the most relevant attributes often requires significant manual effort.

What if users could receive recommendations for interesting ways to sort a table—even when the required attributes are not initially present? These attributes could be automatically sourced from a data lake or even generated using a Large Language Model (LLM) [2].

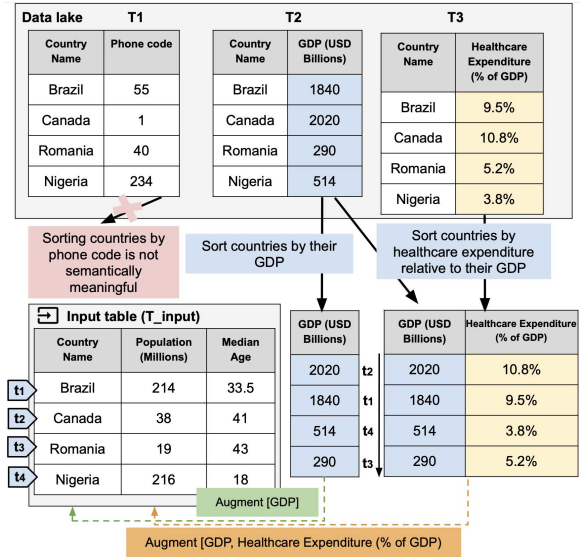


Figure 1: The input table can be augmented with several attributes from the data lake to enable semantically interesting sorting of countries. The tuple IDs are shown next to the suggested augmentation attributes to reflect a descending sort.

Example 1.1. A data scientist, Lou, has an input table of countries with the Population and Median Age attributes (Figure 1). Lou also has access to a data lake containing numerous tables, but it’s unclear which sets of attributes can augment the current table to enable insightful sorting. The data lake tables suitable for augmentation

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 18, No. 12 ISSN 2150-8097.
doi:10.14778/3750601.3750688

must be joinable with the input table. For instance, joining T_1 with the input table using the columns Country Name introduces a new attribute GDP, enabling countries to be sorted by GDP, a crucial measure of economic performance.

Another semantically meaningful way to sort the input table is by joining it first with T_2 and then T_3 , adding two attributes: GDP and Healthcare Expenditure. These attributes allow for sorting the countries first by GDP and then by healthcare expenditure, highlighting how much countries invest in healthcare relative to their GDP. However, not all sorting attributes are useful. For instance, Sorting countries using their phone code is not semantically meaningful.

We present a demo of INSIGHTSORT, an end-to-end system that (1) creates indexes that keep track of the sorting columns in data lake tables; (2) efficiently determines, given an input table, what data lake columns are joinable with it; (3) selects sorting attributes that are semantically meaningful; and (4) diversifies the sorting strategies based on their semantic interestingness. Finally, evaluating LLMs for table sorting is an unexplored area. This demo offers VLDB participants a unique opportunity to assess the performance of several leading LLMs—such as GPT-4o [2], Perplexity AI [9], and Claude [3]—on this important table analysis task.

Related work. Several techniques for join discovery have been proposed [4, 11]. In this line of work, users specify a query (e.g., example table) and the goal is to find joinable tables from the data lake. In Metam [5], the goal is to perform attribute augmentation for a given downstream task by specifying its utility function. INSIGHTSORT differs from existing work because (1) its goal is orthogonal to the join discovery being used, one can choose any join discovery method to find joinable tables; and (2) it searches for semantically-meaningful sorting attributes to augment tables, which is a target task that has never been explored in prior work.

2 SYSTEM OVERVIEW

Figure 2 illustrates the high-level workflow of INSIGHTSORT. In a nutshell, INSIGHTSORT ingests a data lake of tables and builds necessary indexes to navigate it in the offline phase. In the online phase, INSIGHTSORT takes as input a table. INSIGHTSORT finds possible attribute augmentations for the input table from the data lake, scores them by their semantic interestingness, and finally diversifies the sorting results.

2.1 Offline processing of the data lake

In this phase, INSIGHTSORT ingests a data lake of tables and indexes it for joinability testing, i.e., given an input column, which data lake columns are joinable with it.

2.1.1 Building the joinability index. Since the data lake can contain a large number of tables, it is important to build indexes that enable efficient joinability testing. To this end, INSIGHTSORT proceeds as follows:

- (1) We generate vector embeddings for each column in the data lake by following the approach proposed in [4], which leverages the pre-trained language model SBERT [10]. Briefly, this method constructs a textual summary for each column—such as a list

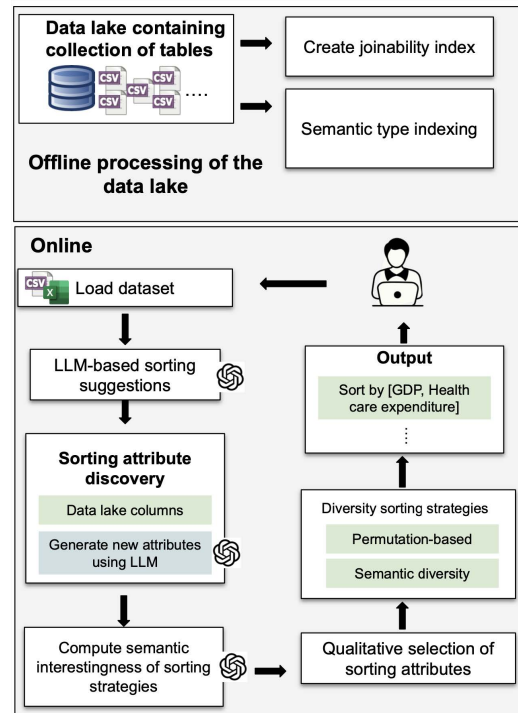


Figure 2: High-level workflow of INSIGHTSORT. INSIGHTSORT ingests and indexes a data lake of tables (offline processing), which is then used in the online phase to discover sorting attributes.

of unique values—and then uses the language model to encode these summaries into vector embeddings.

- (2) After the vector embeddings of all columns have been created, INSIGHTSORT index them using Hierarchical Navigable Small World (HNSW) [7]. This index enables fast Approximate Nearest Neighbor (ANN) queries to check if a query column embedding matches a data lake column embedding within a distance threshold.

2.1.2 Semantic types indexing. INSIGHTSORT creates another HNSW index that indexes the vector embeddings of textual description of columns. To this end, INSIGHTSORT performs the following:

- (1) Because column names are most often not descriptive, for each table, INSIGHTSORT prompts the LLM (GPT-4o [2]) with the attribute names and sample values, and requests the semantic types of each column. The semantic type of a column is its textual description. For instance, a column’s semantic type can be “US universities” if it contains names of US universities.
- (2) INSIGHTSORT uses pre-trained language models (sentence transformers [10]) to generate vector embeddings of the semantic types.
- (3) Finally, INSIGHTSORT indexes those vector embeddings using an HNSW index for fast ANN query answering.

2.2 Online phase

In the online phase, INSIGHTSORT leverages the previously created data lake indexes to find joinable columns to a given query table, and possible join columns for a given sorting description.

2.2.1 Uploading the input table. INSIGHTSORT allows users to upload their input table. INSIGHTSORT will then look for sorting columns by augmenting it with data lake columns or creating new columns using the LLM.

2.2.2 LLM-based sorting suggestions. INSIGHTSORT prompts the LLM with the input table schema, and sample rows, and asks for “interesting” sorting attributes that can be added to the input table. The LLM identifies a list of attributes that can be added to sort the rows. For instance, in the example table *T2* in Figure 1, GPT-4o [2] suggested the sorting attributes in Table 1 (we are only including a sample of them).

2.2.3 Sorting attribute discovery. Given the suggested sorting attributes by the LLM, INSIGHTSORT attempts to locate them in the data lake. Of course, those attributes will not necessarily have the same name as suggested by the LLM, so INSIGHTSORT proceeds as follows:

- (1) **Vector embedding generation:** INSIGHTSORT generates vector embeddings of the suggested sorting attribute textual description using a pre-trained language model. INSIGHTSORT uses Sentence Bert [10] to produce those embeddings.
- (2) **Locating the query attributes:** INSIGHTSORT finds the data lake columns that match the query column by performing an ANN query on the semantic type HNSW index. After this, INSIGHTSORT has to determine if the tables of those matching columns are joinable with the query table. To this end, it uses the joinability index that was created offline.

Additionally, INSIGHTSORT can also ask the LLM to generate a sorting attribute directly. This is appropriate for cases where the sorting attribute is “general-knowledge” and not private (e.g., company-specific data).

Attribute	Description / Use for Sorting
Population (millions)	Sort by population size
GDP per Capita (USD)	Economic comparison across countries

Table 1: A sample of the suggested sorting attributes by GPT-4o for table *T2* in Figure 1.

2.2.4 Computing semantic interestingness of sorting columns. We now define the semantic interestingness of a set of sorting attributes with respect to a query table:

Semantic interestingness of a sorting attribute. Given an input table T and a candidate set of columns $C = \{c_1, c_2, \dots, c_m\}$, the semantic interestingness of a given sort strategy (attributes by which to sort) captures the following two components:

- **Semantic utility.** How semantically interesting the attributes in C are in generating an interesting sort order for T ($\text{semantic_utility}(T, C)$). This is computed by prompting an LLM with the column names of T and C and asking it to derive a score for how

interesting sorting T with C is. This prompt has several components: (1) it frames the LLM as a data expert evaluating the value of sorting a dataset; (2) it provides the dataset’s columns and the specific column(s) proposed for sorting; (3) it asks the LLM to rate the usefulness of the sort based on criteria like analytical value, added insight, and prioritization help; and (4) it instructs the LLM to output only a decimal score between 0 and 1.

- **Column uniqueness.** We also want to make sure the attributes in C do not overlap significantly with the attributes in T . That is, we don’t want to augment the table with duplicate attributes. Column uniqueness is computed by averaging the cosine similarity between the columns in C and the attributes in T (we get the top-1 match from T for each column in C).

In summary, the semantic interestingness is defined as:

$$I(T, C) = \frac{\text{semantic_utility}(T, C) + \text{uniqueness}(T, C)}{2} \quad (1)$$

Seamntic interestingness allows us to determine, for each joinable table in the data lake, whether adding its attributes to the input table would result in semantically-interesting sorting attributes.

2.2.5 Diversifying the results. INSIGHTSORT returns a set of *diverse* sorting columns. That is, we want to make sure we minimize duplicate sorting attributes. To this end, we adopt two strategies:

- **Permutation-based:** Given two sorting attributes A and B , we obtain row permutations S_A and S_B over table T . To ensure diversity, we measure the Hamming or Levenshtein distance between S_A and S_B , requiring it to exceed a threshold θ . Correlated attributes (e.g., GDP and HDI) often yield similar permutations.
- **Semantic diversity:** Here, we diversify based on meaning. INSIGHTSORT (1) generates a textual description of each sort using an LLM, (2) encodes them with a language model, and (3) uses an HNSW index to ensure embedding distances exceed a semantic threshold d .

3 DEMONSTRATION PLAN

We demonstrate INSIGHTSORT on the following real-world datasets: (1) Stackoverflow survey dataset¹; (2) Adult dataset²; and (3) Chicago crime dataset³. As for the data lakes, we will be using a repository of over 1000 CSV files that we crawled from data.gov.

Demonstration outline. In this demo, we aim to showcase (1) the interactive interface of INSIGHTSORT that allows users to explore various table augmentations for different sorting semantics; (2) the importance of interestingness scores in exposing various “interesting” sorting attribute sets; and (3) the system’s diversity sorting which allows users to inspect sorting attributes that are semantically unique. We now present the demonstration scenario:

- ① **Uploading the input table and the data lake.** The participants start by uploading their input table as well as the table repository from which INSIGHTSORT will discover augmentations (Figure 3 ①).
- ② **Customizing joinability thresholds.** Once the data lake has been uploaded, INSIGHTSORT finds joinable columns with the input

¹<https://survey.stackoverflow.co>

²<https://www.kaggle.com/datasets/wenrui/adult-income-dataset>

³<https://www.chicago.gov/city/en/dataset/crime.html>

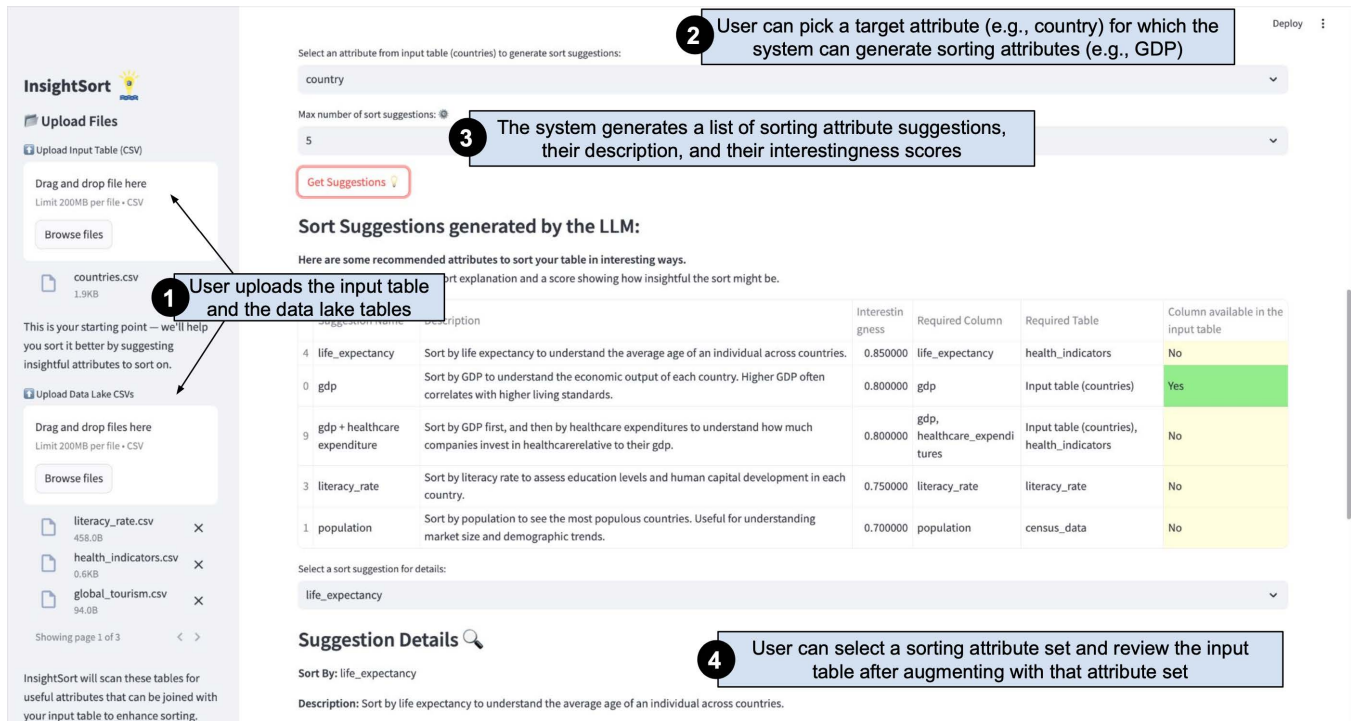


Figure 3: The user interface of INSIGHTSORT. Participants will have the opportunity to discover several sorting attribute sets with different semantics.

table. Participants can adjust the joinability thresholds that are internally used by INSIGHTSORT to determine if two columns are joinable. The goal here is to show that less joinable columns can create several spurious augmentations.

③ **Picking a target attribute.** INSIGHTSORT allows users to pick a particular target attribute (e.g., country) for which INSIGHTSORT will discover sorting attributes (e.g., GDP) as illustrated in Figure 3 ②. Users can also elect not to choose a target attribute, in which case INSIGHTSORT will discover sorting attributes for all possible attributes in the input table.

④ **Sorting attribute suggestions.** INSIGHTSORT discovers a list of sorting attributes from the data lake, and displays their description, as well as their interestingness scores to the user (Figure 3 ③).

⑤ **Input table augmentation.** Upon selecting a particular sorting attribute set suggestion, INSIGHTSORT proceeds to augment the input table with the selected attribute set (Figure 3 ④).

⑥ **Comparing sorting attributes.** INSIGHTSORT allows users to compare various sorting sets based on how unique they are. This uniqueness is determined by how similar the sorting permutes the tuples of the input table, and how semantically unique it is. Users can adjust the thresholds of diversity to explore different sorting suggestions. The goal is to demonstrate that including no diversity filtering results in a large amount of similar sorting suggestions.

⑦ **Comparing different LLMs.** INSIGHTSORT will also feature connections to LLMs other than GPT-4o. Those include Gemini [6], Perplexity AI [9], and [3]. This will allow the participants to select sorting suggestions across different LLMs, which would provide a vast array of options.

Demonstration engagement. In addition to a guided demonstration of INSIGHTSORT, we will encourage participants to upload their own datasets to perform an interactive table exploration session on them. Participants will be able to interactively examine sorting attributes that are semantically relevant to their task and compare them across different LLMs, joinability thresholds, and interestingness scores.

REFERENCES

- [1] [n.d.]. InsightSort demo video. <https://drive.google.com/drive/folders/1Hke2MTkTGF0FRHmrBOWx-nNIFPUcojC9?usp=sharing>
- [2] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv:2303.08774* (2023).
- [3] Anthropic. 2023. *Claude*. <https://www.anthropic.com/index/introducing-claude>
- [4] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyama. 2023. DeepJoin: Joinable Table Discovery with Pre-Trained Language Models. *Proc. VLDB Endow.* 16, 10 (June 2023), 2458–2470. <https://doi.org/10.14778/3603581.3603587>
- [5] Sainyam Galhotra, Yue Gong, and Raul Castro Fernandez. 2023. METAM: Goal-Oriented Data Discovery. *arXiv preprint arXiv:2304.09068* unspecified, unspecified (2023), unspecified–unspecified.
- [6] Google DeepMind. 2023. *Gemini*. <https://deepmind.google/technologies/gemini/>
- [7] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Trans. Pattern Anal. Mach. Intell.* 42, 4 (April 2020), 824–836.
- [8] Fatemeh Nargesian, Erkang Zhu, Renée J. Miller, Ken Q. Pu, and Patricia C. Arocena. 2019. Data lake management: challenges and opportunities. *Proc. VLDB Endow.* 12, 12 (Aug. 2019), 1986–1989. <https://doi.org/10.14778/3352063.3352116>
- [9] Perplexity AI. 2023. *Perplexity AI*. <https://www.perplexity.ai>
- [10] N Reimers. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *arXiv preprint arXiv:1908.10084* (2019).
- [11] El Kindi Rezig, Anshul Bhandari, Anna Fariha, Benjamin Price, Allan Vanterpool, Vijay Gadepally, and Michael Stonebraker. 2021. DICE: data discovery by example. (2021).